

Dokumentation des Geräte Ausleihservice

Name: Leonard Mottke
Matrikel Nr.: m30904

Gliederung:

Planung bis Umsetzung	S3
- Skizzen	S4
- Common	S5
- Server	S6
- Client	S6
Post Mortem	S8
Quellen	S8

Planung bis Umsetzung:

Am Anfang war ich etwas überfordert, wie ich mit der Abgabe anfangen sollte. Ich wollte alles sofort machen und alles irgendwie gleichzeitig starten, aber das hat dazu geführt, dass ich es nicht ganz auf die Reihe bekommen habe. Also habe ich mir einen Plan gemacht, einen einfachen Schritt-für-Schritt-Plan, weil ich keinen Überblick habe, wenn ich zu viel vorhabe. So startete ich mit der einfachsten Aufgabe, nämlich das Anlegen der Module. Ich erstellte einen Klient, einen Server und ein Common-Modul.

Was als Nächstes? Ich habe überlegt, mit dem Client anzufangen, aber ich habe gemerkt, dass das nicht der beste Anfang wäre, also erstellte ich erstmal die Datenbank, da ohne die Datenbank nichts funktioniert. Danach erstellte ich alle Klassen, die in den Modulen gebraucht werden. Schließlich beschloss ich, mit der einfachsten Aufgabe anzufangen, nämlich das Common-Modul fertigzustellen. Das Common-Modul ist eine Verbindungsstelle zwischen Klient und Server, dort werden die Geräte definiert.

Als Nächstes habe ich den Server aufgesetzt und dafür gesorgt, dass dieser sich mit der Datenbank verbindet. Es lief ganz gut, bis mir aufgefallen ist, dass der Server sich sofort beendet, gleich nachdem er aufgerufen wurde. Das wurde dann so umgeändert, dass der Server zwar läuft, aber er konnte nicht mehr gestoppt werden, was bis jetzt auch noch der Fall ist. Deshalb muss der Server in IntelliJ gestoppt werden.

Nachdem das Common-Modul und der Server fertig waren, habe ich mich am Klient weitergearbeitet. Ein Kommilitone hat mir das Programm SceneBuilder empfohlen, damit ich die FXML-Dateien nicht im bei mir abstürzenden IntelliJ-Programm erstellen musste. Es ging recht gut voran. Schritt für Schritt habe ich die FXML-Dateien erstellt und die Fenster bearbeitet.

Ich hatte verschiedene Ideen, wie ich die Fenster gestalten sollte. Im Folgenden werde ich ein paar Beispielskizzen zeigen, die ich ursprünglich im Kopf hatte. Letztlich habe ich mich jedoch etwas anders entschieden, wie man auch im Benutzerhandbuch nachlesen kann.

Suchen  admin Kunde		
Gerät	Änder	⊖
Gerät	Änder	⊖
Gerät	Änder	⊖
Gerät	Änder	⊖
Gerät	Änder	⊖
Gerät	Änder	⊖
Gerät	Änder	⊖

Suchen admin Kunde		
Gerät	ausleihen	
Gerät	ausleihen	
Gerät	ausleihen	
Gerät	zurückgeben	02.05.75
Gerät	auslei!	
Gerät	zurück	
Gerät	zurück	05.15.75

Geräte hinzufügen

Name:

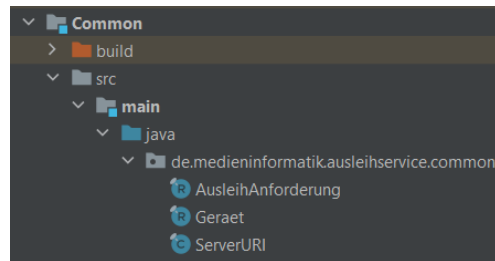
Geräte ändern

Name:

Kunde / Admin

Name:

Das Common hat drei Klassen:



Die Ausleihanforderung: Hier können die Kunden die Geräte ausleihen.

```
public record AusleihAnforderung(  
    3 usages  
    String gerateNamen,  
    2 usages  
    String kundenName  
) {  
  
}
```

Das Gerät: Es wird ein Gerät definiert.

```
public record Geraet(  
    17 usages  
    String geraetName,  
    4 usages  
    String ausgeliehenVon,  
    5 usages  
    Date rueckgabeDatum  
) {  
  
}
```

Die ServerURI: Dient dazu, sich mit dem Server zu verbinden.

```
public final class ServerURI {  
    /**  
     * Privater Konstruktor, um die Instanziierung dieser Utility-Klasse zu verhindern.  
     */  
    private ServerURI(){}  
    /**  
     * Vordefinierte URI für den Server.  
     */  
    1 usage  
    private static final URI uri = UriBuilder.newInstance()  
        .scheme("http")  
        .host("localhost")  
        .port(8080)  
        .path("geraetesammlung")  
        .build();  
    /**  
     * Gibt die vordefinierte Server-URI zurück.  
     * @return die vordefinierte {@link URI} des Servers  
     */  
    3 usages  
    public static URI getUri() { return uri; }  
}
```

Der Server hat vier Klassen:

Main: Startet den Server.

Datenbankverbindung: Stellt die Verbindung zur Datenbank her.

EndPunktServer: Der Einstiegspunkt für den Klient.

EndPunktApplication: Startet die gesamte Anwendung, sodass auf den Server zugegriffen werden kann.

Code Beispiel für die EndpunktApplication:

```
public class EndpunktApplication extends Application {
    2 usages
    private final Set<Class<?>> classes = new HashSet<>();

    /**
     * Konstruktor der Klasse.
     * Registriert die Klasse {@link EndPunktServer} als REST-Endpunkt.
     */
    public EndpunktApplication() {classes.add(EndPunktServer.class); }

    /**
     * Gibt die registrierten Klassen zurück, die als REST-Endpoints fungieren.
     * @return Ein Set der registrierten Klassen für JAX-RS.
     */
    @Override
    public Set<Class<?>> getClasses() {return classes; }
}
```

Der Client hat drei Klassen für die Anwendung selbst und vier Controller-Klassen:

Main: Startet die Client-Anwendung.

ClientApplication: Startet den Anmelde-Controller.

Serververbindung: Verbindet den Client mit dem Server.

Die vier Controller-Klassen:

AnmeldeController: Öffnet das Anmeldefenster.

HauptseitenController: Ermöglicht das Ausleihen von Geräten. Sowohl hier als auch im AusleihseitenController gibt es eine zusätzliche Extends-Klasse.

AusleihseitenController: Steuert die Ausleihseite.

AdminController: Ermöglicht das Hinzufügen, Löschen und Bearbeiten von Geräten.

Ein Code beispiel, Main:

```
public class Main {  
  
    /**  
     * Der Hauptteil der Anwendung. Startet die JavaFX-Anwendung, die den Klienten startet.  
     * @param args die Befehlszeilenargumente (werden in diesem Fall nicht verwendet)  
     */  
    public static void main(String[] args) {  
        Application.launch(KlientApplication.class);  
    }  
}
```

In der Main-Klasse habe ich verschiedene kleine Tests durchgeführt, um zu überprüfen, ob alles richtig funktioniert. Diese Tests habe ich später auskommentiert.

```
//ServerVerbindung.ausleihen(new AusleihAnforderung("Todesstern","admin"));  
//ServerVerbindung.geraetAendern("Traktor","Tracktor");  
//ServerVerbindung.geraetHinzufuegen(new Geraet("Inator",null));  
//ServerVerbindung.geraetLoeschen("Todesstern");
```

Post Mortem:

Insgesamt lief das Projekt ganz gut, aber es gab auch einige Herausforderungen. Die größte Schwierigkeit war am Anfang, dass ich zu viel auf einmal machen wollte und mich verzettelt habe. Erst durch einen strukturierten Plan konnte ich besser arbeiten. Auch die Probleme mit dem Server haben etwas Zeit gekostet, aber am Ende hat es irgendwie funktioniert. Was ich aus diesem Projekt gelernt habe, ist, dass eine gute Planung sehr wichtig ist. Außerdem habe ich gemerkt, dass es hilft, sich auf eine Aufgabe nach der anderen zu konzentrieren, anstatt alles gleichzeitig zu versuchen.

Am Ende bin ich zufrieden damit, wie das Projekt gelaufen ist. Es hat zwar ein paar Probleme gegeben, aber ich konnte sie nach und nach lösen. Beim nächsten Mal werde ich versuchen, von Anfang an strukturierter vorzugehen und mir direkt einen klaren Plan zu machen.

Quellen:

Ich habe mich am Skript orientiert und auch noch einmal mit den Tutoren gesprochen. Außerdem habe ich die Kommentare teilweise mit ChatGPT geschrieben und es zur Rechtschreibkorrektur genutzt.